

Technical Interview Prep — Amii × ZeroKey ML Resident

Job ID: 1085 · **Round:** technical (after the Erika Bruneau pre-screen) **Likely interviewer:** Xu Han, Machine Learning Scientist, Amii Advanced Technology (see companion guide §1). ZeroKey staff may also join. **Companion guide:** [amii-zerokey-ml-resident.md](#) covers company, role and pre-screen material. **Prepared:** 2026-09-25 · **Revised:** 2026-09-28, to match the recruiter's format email

The format (from the recruiter's email)

60 minutes, focused on "concepts and problems of machine learning and related mathematical methods for analyzing motions." There is no coding.

Part	What	Weight	Where it's covered
1. Technical questions	Main topics: fundamentals of ML and statistics; modern deep learning for time series, computer vision and related areas; practical industry tasks involving time series and computer vision	~40–50 min	§1–§5
	Minor topics: basic modelling approaches for spatial movement; basics of AI agents		§6–§7
2. Mini case study	Shown a short case in real time and asked to describe your approach. 10–20 min. Interactive: both sides ask questions. Any tools allowed, not required.	10–20 min	§9–§10

What they're testing: "both fundamentals and critical thinking in AI/ML."

What changed from the earlier version of this guide:

- **No coding.** Explain methods clearly with equations and diagrams, not code. Reference code is in Appendix A, for understanding only.
- **Statistics is a main topic.** New §1.2.
- **Computer vision is a main topic.** New §4. Expect questions beyond RTLS: detection, pose, video, industrial inspection.
- **RL isn't on the list.** It's now a short just-in-case section (§8). Glassdoor reported RL questions in other Amii rounds, but this email doesn't mention it.
- **Agents are a minor topic.** Basics only (§7).
- **Motion analysis is in the headline sentence.** Kalman filtering, kinematics, trajectory similarity and pose-based analysis stay **Tier 1** (§6), even though "spatial movement" is listed as minor.
- **The case study** now has a full playbook and practice cases (§9–§10).

Study priority

Tier	Topic	Section
1	Case-study playbook plus rehearsing 2–3 practice cases out loud	§9–§10
1	ML and statistics fundamentals	§1
1	Deep learning for time series	§3
1	Deep learning for computer vision (pose, video, detection, industrial inspection)	§4
1	Mathematical methods for motion (kinematics, Kalman, DTW, trajectory similarity)	§6

Tier	Topic	Section
1	Defending your own projects	§11
2	Practical industry tasks (time series + CV)	§5
2	Data handling and evaluation for time series and video	§2
2	AI agent basics	§7
3	RL (just in case), physics-informed ML	§8, §6.9

Two honesty rules for the whole interview:

1. **F1 = 0.76** (JAES, 14 ms on RPi4). Never 90%. If they quote your cover letter's ">90%" back to you, correct it yourself.
2. **Kalman filtering**: claim what you've actually done. §6 makes you fluent. Fluent is not the same as experienced, so don't present it as experience.

How to explain without code: state the idea in one sentence → write the key equation → give a concrete example on ZeroKey-like data → name one limitation. For example: "A Kalman filter is a recursive Bayesian estimator. Predict with a motion model, correct with the measurement, weighted by the gain $K = P^T H^T (H P^T H^T + R)^{-1}$. On RTLS it mainly gives velocity estimates and outlier rejection. It assumes linear dynamics and Gaussian noise."

Part A — Technical questions

1. Fundamentals of ML and statistics [main topic]

1.1 Core ML concepts

- **Bias–variance decomposition**: $E[(y - \hat{y})^2] = \text{bias}^2 + \text{variance} + \text{irreducible noise}$. Underfitting = high bias. Overfitting = high variance.
- **Diagnosing**: learning curves. Train and validation error both high → underfitting (more capacity, better features). Train low, validation high → overfitting (more data, regularization, simpler model). **In time series, a suspiciously high validation score usually means leakage, not skill** (§2.2).
- **Regularization**:
 - L2 is the same as a **Gaussian prior** on the weights (MAP estimation); L1 is a **Laplace prior**, which gives sparsity.
 - Dropout works like approximate ensembling. Early stopping, data augmentation and weight sharing are also regularizers.
- **Generalization**: train/validation/test sets, cross-validation, the curse of dimensionality, and the "no free lunch" idea (which inductive bias fits the data).
- **Optimization**:
 - Gradient descent and SGD; momentum; Adam; AdamW (decoupled weight decay).
 - Learning-rate schedules: warmup, cosine decay.
 - Vanishing and exploding gradients: residual connections, normalization, gated RNNs, gradient clipping.
- **Loss functions**:
 - MSE is the Gaussian likelihood; MAE is the Laplace likelihood and is robust to outliers; Huber sits in between.
 - Cross-entropy is the negative log-likelihood of a categorical distribution.

- Focal loss down-weights easy examples, which helps with class imbalance.
- **Classical models, and when they still win:**
 - Linear and logistic regression: interpretable, strong baselines.
 - SVMs: maximum margin, kernel trick.
 - Decision trees → **random forest** (bagging; reduces variance) → **gradient boosting** (fits residuals sequentially; reduces bias). XGBoost and LightGBM are often best on tabular or engineered features.
 - k-NN, k-means, Gaussian mixtures fitted with **EM**.
- **Dimensionality reduction:** PCA takes the eigenvectors of the covariance matrix (equivalently, the SVD of centred data) and keeps the directions of largest variance. t-SNE and UMAP are for visualization only; their distances aren't meaningful.
- **Information theory:**
 - Entropy $H(p) = -\sum p \log p$.
 - Cross-entropy $H(p,q)$.
 - **KL divergence** $D_{KL}(p||q) = \sum p \log(p/q)$: not symmetric, always ≥ 0 .

1.2 Statistics [main topic, new]

- **Probability basics:**
 - Bayes' rule: $p(\theta|x) = p(x|\theta)p(\theta)/p(x)$. Conditional independence. Law of total probability.
 - **Base-rate fallacy:** a 99%-accurate detector for an event that happens 0.1% of the time still produces mostly false alarms. **Very relevant to rare risk events.**
- **Distributions you should be able to name and use:**
 - Gaussian (sensor noise), multivariate Gaussian with **Mahalanobis distance** (outlier gating).
 - Bernoulli and binomial (detection outcomes).
 - **Poisson** (number of events or alerts per shift); **exponential** (time between events).
 - χ^2 (sum of squared standard normals; used for Kalman innovation gating).
- **Estimation:**
 - **MLE** maximizes the likelihood. For a Gaussian, the MLE of the mean is the sample mean, and the MLE of the variance divides by n , which is biased; dividing by $n-1$ is unbiased.
 - **MAP** is MLE plus a prior, which is where regularization comes from.
 - An estimator's bias, variance and consistency.
- **Central limit theorem and standard error:** $SE = \sigma/\sqrt{n}$. Confidence intervals: "95% of intervals built this way contain the true value." That's not the same as a Bayesian credible interval.
- **Bootstrap:** resample to get confidence intervals on any metric, e.g. F1 on a small test set. **For time series, use a block bootstrap**, because ordinary resampling breaks the autocorrelation.
- **Hypothesis testing:**
 - The null hypothesis. The **p-value** is the probability of data at least this extreme *if the null is true*. It is *not* the probability that the null is true.
 - Type I errors (false positives, rate α) and Type II errors (false negatives, rate β). **Power** = $1 - \beta$.
 - **Multiple comparisons:** Bonferroni (controls the family-wise error rate) or Benjamini-Hochberg (controls the false discovery rate). Relevant when you monitor many cells or workers at once.
 - **Comparing two models on the same test set:** McNemar's test (paired classification errors), or a paired t-test / Wilcoxon test on per-fold scores.
 - Your experience: user-study statistics ($p < .05$ in C2), Youden's index for thresholding.

- **Autocorrelation matters:** time-series samples aren't independent, so the **effective sample size is smaller** than the raw count, naive p-values and confidence intervals come out too narrow, and random train/test splits leak information.
- **Correlation vs. causation:**
 - Confounders: shift, worker experience, product mix.
 - **Simpson's paradox:** a trend can reverse once you split by a confounder.
 - For "did our intervention reduce risk?", a before/after comparison is confounded by other changes. Use **control cells** and **difference-in-differences**, or staggered rollouts.
- **Linear regression assumptions:** linearity, independent errors, constant variance (homoscedasticity), normally distributed residuals (needed for inference). Autocorrelated residuals in time series break the independence assumption.
- **Time-series statistics:**
 - **Stationarity** (constant mean and variance over time); differencing to remove trends.
 - ACF and PACF.
 - ARIMA as a classical baseline; seasonality (shift and weekly cycles).
 - Spectral density, for repetitive motion.
- **Evaluation metrics:**
 - Confusion matrix; precision, recall, F1.
 - **ROC vs. PR curves:** under heavy class imbalance, prefer PR-AUC, because ROC curves look optimistic.
 - Calibration: reliability diagrams, Brier score, temperature scaling.
 - Regression errors: MAE, RMSE (penalizes large errors more), MAPE (unstable near zero).

1.3 Class imbalance (risky events are rare)

- Use metrics that expose it: PR-AUC, precision and recall at the operating point, **false alarms per shift**, detection delay.
- Fixes: class weights or focal loss; resampling at the *event* level; hard-negative mining; switching to an anomaly-detection framing when positives are nearly absent.
- Thresholds follow from the cost of errors. Youden's J maximizes TPR – FPR, but factories usually fix an **alarm budget** (e.g. "at most 2 false alerts per shift") and choose the threshold to meet it.

2. Data handling and evaluation for time series and video

2.1 What the RTLS data probably looks like

`(tag_id, t, x, y, z, [quality])` for tags on workers (badge, wrist), tools, bins, robots and AGVs. ZeroKey's public figures: **±1.5 mm accuracy, 10,000+ events/s, anomaly detection in under 500 ms**. The update rate per tag isn't public, so **ask** in the case study.

Pre-processing:

- Align everything to one clock.
- **Irregular sampling:** resample, or use models that take timestamps as input.
- **Dropouts and occlusion:** ultrasound needs a clear path, so bridge short gaps with Kalman prediction, flag long gaps, and never interpolate across long gaps.
- **Multipath outliers:** reject with physics limits and innovation gating.

- **Coordinate frames:** convert facility coordinates to work-cell coordinates so models transfer between cells.
- **Context joins:** zones, SOP steps, MES and torque events, and which tag belongs to which entity.

2.2 Leakage in time series and video (a strong answer; most candidates miss it)

- **Overlapping windows** with random splits put near-duplicate windows in both train and test. Split **by time block, session, worker, cell or site**, and leave a gap between splits.
- **Video:** frames from the same clip are near-duplicates, so split **by video, camera or site**, never by frame.
- **Group k-fold** by worker or site tests generalization to new people and places, which is the real deployment question.
- **Rolling-origin (forward-chaining) validation** for forecasting.
- **Fit scalars, PCA and so on inside each training fold.**
- Watch for **label leakage:** features that indirectly encode the label.

2.3 Evaluating at the event level

- Frame-level metrics count windows or frames. Operators experience *events*.
- An event counts as **detected** if an alert overlaps it within a tolerance. An alert that overlaps no event is a **false alarm**. Report **detection delay** as well.
- For segmentation-style tasks, use segmental F1 at an IoU threshold (standard in temporal action segmentation) and edit score.

2.4 Distribution shift

- Types: covariate shift (new layout, new camera angle, different lighting), label shift (different base rates), concept drift (the SOP changes).
- Responses: invariant features, per-site calibration, a small labelled set for each new site, domain adaptation, monitoring input distributions (PSI or KL on feature histograms), and retraining triggers.

2.5 Augmentation

- **Trajectories:** rotation about the vertical axis, translations, **time-warping and speed scaling**, jitter noise matched to the sensor, random dropouts, mirroring (only where physically valid).
- **Images and video:** crops, flips (careful: flipping changes left/right semantics), colour and brightness jitter (factory lighting varies), blur, cutout, mixup/cutmix, temporal jitter (shifting a clip's start), frame-rate changes.
- **Synthetic data:** your rule-based variation generator (about 10k variations per pattern in JAES). The risk is **generator bias:** always test on real, held-out data.

3. Modern deep learning for time series [main topic]

3.1 Model families (be ready to choose one and defend it)

Model	Idea	Strengths	Weaknesses	Fit at ZeroKey
Features + GBDT	Hand-crafted window features → boosted trees	Strong, interpretable, tiny	Features are manual	First baseline; explaining results to engineers

Model	Idea	Strengths	Weaknesses	Fit at ZeroKey
RNN / LSTM / GRU (your core)	Recurrent state; gates control what's remembered	Streaming with constant memory per step, low latency, small data	Weaker long-range memory, sequential training	On-device streaming detection
TCN	Dilated causal 1D convolutions	Parallel training, fixed receptive field, quantizes well	Fixed context length	Edge streaming; good latency/accuracy balance
Transformer	Self-attention over time steps	Long context, multimodal fusion, pre-training	$O(T^2)$ cost, needs data, harder on the edge	Server-side analysis, fusion
Patch-based transformers (PatchTST)	Groups of time steps become tokens	Much cheaper; strong at forecasting	Patch size is a design choice	Long-horizon forecasting
Graph networks (ST-GCN)	Graph convolution across entities + temporal convolution	Model <i>interactions</i>	Graph design	Worker↔robot↔tool risk
PointNet / Deep Sets	Encoder that ignores order	Variable, unordered set of tags	No temporal structure on its own	Per-frame encoding of all tags in a cell
Temporal segmentation (MS-TCN)	Multi-stage refinement of per-frame labels	Step boundaries over long sequences	Needs frame-level labels	SOP step segmentation
Neural ODEs / latent ODEs	Learned continuous-time dynamics	Irregular sampling	Slow, can be finicky	Irregular RTLS streams
HMM / HSMM	Hidden steps with duration models	Explicit structure, few labels	Limited expressiveness	SOP steps with duration priors
Time-series foundation models (Chronos, TimesFM, Moirai, MOMENT)	Pre-trained on huge corpora	Zero-shot forecasting and embeddings	Not built for 3D multi-entity motion	Baselines and feature extractors

Your default answer: "Start with engineered features plus GBDT, and a state machine or DTW for sequence compliance, as interpretable baselines. Then a causal TCN or GRU for streaming detection at the edge, and a transformer or graph model on the server for cross-entity interaction. Choose by benchmarking under the latency budget, the way I compared RNN and DTW on the RPi4."

3.2 Explaining key concepts without code

- **LSTM:** input, forget and output gates plus a cell state. The additive path through the cell state keeps gradients from vanishing. **GRU** merges the gates into update and reset gates: fewer parameters, similar performance.
- **Attention:** $\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k}) \cdot V$.
 - Each time step scores every other step and takes a weighted sum of their values.
 - $\sqrt{d_k}$ keeps dot-product variance near 1 so softmax doesn't saturate.
 - **Multiple heads** learn different relationships.
 - **Positional encoding** is needed because attention ignores order. For irregular timestamps, encode actual time (e.g. Time2Vec).
 - **Causal masking** for streaming; a **KV cache** makes each new step cost $O(T)$.
- **Transformer block:** LayerNorm → multi-head attention → residual → LayerNorm → feed-forward → residual (pre-norm, which trains more stably).

- **Why you chose RNNs in your PhD [YOU]:** constant compute per step gave predictable latency on the RPi4, and the dataset was small. Result: **14 ms, 31.4% CPU, F1 0.76** vs. DTW's 1038 ms, 48.9% CPU, F1 0.65. What you'd try now: a TCN and a small causal transformer, benchmarked under the same budget.

3.3 Learning with few labels

- **Self-supervised pre-training:** masked reconstruction, next-step prediction, contrastive learning (e.g. TS2Vec). ZeroKey has far more unlabelled data than labelled data.
- **Weak supervision:** SOP definitions, MES/torque events and geofence rules as noisy labels.
- **Active learning:** ask engineers to label only the most uncertain cases.
- **Synthetic data and simulation,** with sim-to-real checks.
- **Transfer learning** from related sensors or tasks, e.g. HAR (human activity recognition) datasets from wearable IMUs.

4. Modern deep learning for computer vision [main topic, new]

Your background is **industrial classical CV** (MAS: OpenCV template and feature matching, registration and warping, camera arrays, PLC integration). Modern deep CV is your lighter area, so know the landscape well enough to reason about it, and be upfront that your hands-on CV was classical.

4.1 Backbones

- **CNNs:**
 - Convolution means local receptive fields with shared weights, which gives translation equivariance. Pooling and stride downsample.
 - **ResNet** (residual connections made very deep networks trainable) → EfficientNet (compound scaling) → **ConvNeXt** (a modernized CNN that competes with ViTs).
- **Vision Transformer (ViT):** splits the image into patches that become tokens, then applies self-attention. It has weaker built-in inductive bias, so it needs lots of data or pre-training. Swin Transformer uses hierarchical windowed attention.
- **Self-supervised and foundation models:**
 - **DINO/DINOv2** give strong general features without labels.
 - **MAE** pre-trains by reconstructing masked patches.
 - **CLIP** learns an image-text embedding space by contrastive training, which enables zero-shot classification.
 - **SAM** does promptable segmentation.
- **Transfer learning:** a linear probe (frozen backbone) → fine-tuning the last layers → full fine-tuning. The more target data you have, the more layers you unfreeze.

4.2 Detection and segmentation

- **Detection:**
 - Two-stage: Faster R-CNN (region proposals, then classification).
 - One-stage: **YOLO family**, RetinaNet (introduced **focal loss** for imbalance).
 - Set prediction: **DETR** (a transformer with Hungarian matching; no NMS needed).
 - Concepts: **IoU**, **non-maximum suppression**, mAP@IoU.
- **Segmentation:**

- Semantic (every pixel gets a class): **U-Net** (encoder–decoder with skip connections), DeepLab.
- Instance: **Mask R-CNN**.
- Promptable: SAM.

4.3 Human pose estimation (the most relevant CV topic for this role)

- **2D pose:**
 - **Top-down:** detect each person, then predict keypoint heatmaps (HRNet, ViTPose). More accurate, and cost grows with the number of people.
 - **Bottom-up:** detect all keypoints, then group them (OpenPose uses part affinity fields). Cost is roughly independent of the number of people.
 - **MediaPipe / BlazePose:** light and on-device.
- **3D pose:** lift 2D keypoints to 3D with a temporal network (VideoPose3D), triangulate from multiple calibrated cameras, or use depth sensors.
- **Metrics:** PCK (percentage of correct keypoints), OKS-based AP (COCO), MPJPE (3D error in mm).
- **Failure modes in factories:** occlusion by machines and parts, unusual postures, gloves and PPE, poor lighting, a single camera viewpoint (depth ambiguity).
- **Link to ZeroKey:** pose estimation outputs **keypoint trajectories**. RTLS gives a few tag trajectories with mm accuracy and no occlusion from lighting. **The downstream motion analysis is the same** (action recognition, ergonomics, SOP compliance). Vision gives full-body coverage; RTLS gives precision and privacy.

4.4 Video understanding and action recognition

- **Two-stream networks:** RGB frames plus **optical flow** (motion).
- **3D CNNs:** C3D, I3D (2D ImageNet filters inflated into 3D).
- **Efficient options:** **SlowFast** (a slow path for appearance, a fast path for motion), **TSM** (temporal shift module: time modelling in a 2D CNN at almost no extra cost).
- **Video transformers:** TimeSformer (divided space-time attention), ViViT, **VideoMAE** (self-supervised).
- **Skeleton-based action recognition:** **ST-GCN** on pose sequences. Lighter, more robust to appearance changes, and better for privacy than raw video. **This is what's closest to RTLS data.**
- **Temporal action localization and segmentation:** finding *when* actions happen in long videos (MS-TCN, ActionFormer).

4.5 Tracking and motion in video

- **Optical flow:** Lucas–Kanade (sparse; assumes brightness constancy and small motion), Farnebäck (dense), **RAFT** (deep, iterative refinement).
- **Multi-object tracking:**
 - **SORT = Kalman filter** for motion prediction + **Hungarian matching** on IoU between predicted and detected boxes.
 - DeepSORT adds appearance embeddings for re-identification. ByteTrack also associates low-confidence detections.
 - **A good link:** "SORT is a Kalman filter plus assignment, the same state-estimation machinery I'd use on RTLS tracks."

4.6 Camera geometry basics

- **Pinhole model:** $x = K[R|t]X$. K holds the intrinsics (focal length, principal point). $[R|t]$ are the extrinsics (camera pose).
- Lens distortion correction; calibration with checkerboards.
- **Homography** maps between planes (e.g. a floor plane to a top-down view).
- **Stereo and epipolar geometry:** depth from disparity. Multi-view triangulation.
- Your Promptly work was **image registration and warping** from camera-captured garment geometry, which is applied geometric CV [YOU].

4.7 Industrial CV in practice (connects to your MAS work and Xu Han's CV quality-inspection work [XH])

- **Defect detection with few defect examples:**
 - Unsupervised anomaly detection trained on good parts only: **PatchCore** (memory bank of patch features), **PaDiM**, autoencoders.
 - Benchmark: **MVTec AD**.
- **What actually matters:**
 - Controlled **lighting** and optics often matter more than the model (you learned this with microscopic camera arrays).
 - Fixtures and consistent part placement.
 - Explaining decisions to QA staff: **Grad-CAM** heatmaps, or your overlay approach.
 - A low **false-reject rate** (rejecting good parts is expensive).
 - Drift between lines and suppliers.
 - Integration with the **PLC** and reject mechanism.
- **Classical vs. deep:** classical CV is deterministic, explainable, and needs little data, which suits tightly controlled setups. Deep learning handles variation and open-ended defect types. Hybrids are common.

5. Practical industry tasks: time series and CV [main topic]

For each task: how you'd frame it, the baseline, the model, and the catch. Have one short example from your experience ready for each.

Task	Framing	Baseline → model	Catch	Your evidence
Human activity / action recognition (IMU, RTLS, video)	Classify windows or segment streams	Features + GBDT → CNN/GRU/TCN; pose → ST-GCN	Leakage across people; label boundaries are fuzzy	McGill IMU gesture detection (F1 0.78)
Anomaly detection	Mostly unlabelled; the definition of "normal" drifts	Z-score/EWMA/isolation forest → autoencoder/forecast residual	Anomaly ≠ problem; thresholds; alert fatigue	Forestpin anomaly scoring; MAS monitoring
Predictive maintenance	Remaining useful life (RUL) or failure within horizon H	Spectral features (vibration) + GBDT → sequence models; survival analysis	Very few failures; censored data	MAS machine IoT

Task	Framing	Baseline → model	Catch	Your evidence
Forecasting (cycle time, throughput, demand)	Multi-horizon	Seasonal naïve / ARIMA → GBDT with lags → TFT, PatchTST, foundation models	Leakage; changing regimes	—
Visual quality inspection	Classification, detection or segmentation of defects	Rules and templates → CNN / PatchCore	Lighting, false rejects, rare defect types	MAS care-label QC, fabric defects
Process / SOP compliance	Is the sequence correct?	State machine / DTW → HMM → learned segmentation	Legitimate variations of the order	McGill hierarchical FSM, PhD pattern detection
Multi-sensor fusion	Align streams, then fuse early, mid or late	Late fusion of separate detectors → joint models	Clock sync, missing modalities	MUSMET audio + EEG + XR tracking sync
Edge deployment	Latency and power budgets	Profile → smaller architecture → quantize	Measuring end to end, not FLOPs	14 ms on RPi4

5.1 The workflow for an industrial ML project (say it as a lifecycle)

1. **Frame the problem with stakeholders.** What decision changes? What does an error cost? What's the return on investment?
2. **Audit the data:** coverage, noise, labels, biases.
3. **Define labels and a taxonomy** with domain experts.
4. **Baseline**, then **evaluation harness** (leak-free splits, event-level metrics).
5. Iterate on models.
6. **Shadow deployment:** the model scores silently alongside the current process.
7. **Pilot**, then full rollout.
8. **Monitor:** drift, alert rates, operator feedback.
9. **Retrain.**
10. **Hand over:** documentation and knowledge transfer. This is the Amii mandate.

5.2 Edge deployment (explain conceptually)

- **Quantization:** map floats to int8 with a scale and a zero-point, $q = \text{round}(x/s) + z$.
 - PTQ calibrates ranges on sample data after training.
 - QAT simulates quantization during training.
 - Weights use per-channel scales; activations use per-tensor scales.
 - Keep sensitive layers in higher precision (first and last layers, softmax, normalization, recurrent state).
- **Other techniques:** structured pruning, **knowledge distillation** (a big server model teaches a small edge model), choosing an architecture that fits the budget from the start.
- **Runtimes:** ONNX → ONNX Runtime, **TensorRT** (NVIDIA Jetson; ZeroKey is in NVIDIA Inception), TFLite.
- **Streaming state:** carry the RNN or TCN state between calls instead of recomputing windows. That's how you got 14 ms.
- **Profile end to end** (p95, not the mean): sensor → position → filter → features → model → post-processing.

5.3 MLOps essentials

- Reproducible training (fixed seeds, versioned data).
 - Tests on feature code, e.g. a Kalman filter on synthetic trajectories with known ground truth.
 - A frozen regression-evaluation set; a model registry.
 - Shadow → canary → full rollout.
 - Drift monitoring per site; latency SLOs; operator feedback ("false alarm" button).
 - Your experience: CI/CD and code review at MAS, Docker, Airflow, Jenkins [YOU].
-

6. Mathematical methods for analyzing motion [headline topic, and "spatial movement" as a minor one]

The email's headline is "mathematical methods for analyzing motions." Expect questions here even though it's also labelled minor.

6.1 Kinematics, and why differentiating noise is dangerous

- Position → velocity (first derivative) → acceleration (second) → **jerk** (third; a smoothness measure).
- Finite differences amplify noise. With position noise σ and sample interval Δt :
 - velocity noise $\approx \sqrt{2} \cdot \sigma / \Delta t$
 - acceleration noise $\approx \sqrt{6} \cdot \sigma / \Delta t^2$
 - Example: $\sigma = 1.5$ mm at an *assumed* 20 Hz ($\Delta t = 50$ ms) gives velocity noise $\approx$ **42 mm/s** and acceleration noise $\approx$ **1.5 m/s²**, which is large compared with real motion.
- **So smooth before differentiating**, or estimate derivatives inside the state vector.
- Other kinematic features: speed, path length, curvature, turning angle, dwell time, straightness index.

6.2 Smoothing and filtering

- Moving average (lags and blurs peaks); median filter (removes spikes).
- **Savitzky–Golay** (local polynomial fit; keeps peaks; gives derivatives directly).
- **Butterworth low-pass**. Zero-phase filtfilt is offline only. Causal filters add **group delay**, which matters for real-time alerts.
- **Kalman filter** (causal, model-based) and the **RTS smoother** (a backward pass that gives the best offline trajectory; good for building training labels).

6.3 The Kalman filter (explain on a whiteboard; no code needed)

- **State (3D, constant velocity):** $x = [p, v]^T$.
- **Model:** $x_k = F \cdot x_{k-1} + w$, with $F = \begin{bmatrix} I, \Delta t \cdot I \\ 0, I \end{bmatrix}$ and $w \sim N(0, Q)$.
- **Measurement:** $z = H \cdot x + v$, with $H = [I \ 0]$ and $v \sim N(0, R)$.
- **Predict:** $\hat{x}^- = F \cdot \hat{x}$ and $P^- = F \cdot P \cdot F^T + Q$.
- **Update:**
 - innovation $y = z - H \cdot \hat{x}^-$
 - $S = H \cdot P^- \cdot H^T + R$
 - $K = P^- \cdot H^T \cdot S^{-1}$

- $\hat{x} = \hat{x}^- + K \cdot y$
- $P = (I - K \cdot H) \cdot P^-$
- **Intuition:** the gain K balances trust in the model (Q) against trust in the sensor (R). At ± 1.5 mm, R is tiny, so the filter mostly **estimates velocity, bridges dropouts and rejects outliers** rather than smoothing position.
- **Choosing Q :** use the white-noise-acceleration model $Q = \sigma_a^2 \cdot G \cdot G^T$ with $G = [\frac{1}{2}\Delta t^2 \cdot I; \Delta t \cdot I]$. Use a different σ_a per tag type, since hands accelerate much faster than AGVs.
- **Choosing R :** from the sensor spec, or from the variance of a stationary tag.
- **Validation:** the normalized innovation squared (NIS) should follow a χ^2 distribution.
- **Why it's optimal:** it's the exact Bayesian posterior for linear-Gaussian systems, and the best *linear* estimator otherwise.
- **Outlier gating:** reject a measurement if $d^2 = y^T S^{-1} y > \chi^2_3(0.99) \approx 11.34$.
- **Variants:**
 - **EKF:** linearizes a non-linear model with Jacobians (e.g. raw ranges to anchors).
 - **UKF:** propagates sigma points; no Jacobians needed.
 - **Particle filter:** handles multimodal, non-Gaussian cases.
 - **IMM:** switches between motion models (standing, walking, reaching).
 - **KalmanNet:** learns the gain with an RNN while keeping the model structure $[XH]$.
- **Link to vision:** SORT tracking is a Kalman filter plus Hungarian assignment (§4.5).

6.4 Comparing trajectories and matching templates

- **Euclidean distance** (requires equal length and alignment).
- **DTW:** finds the lowest-cost monotonic alignment, $O(nm)$.
 - A **Sakoe–Chiba band** limits warping.
 - **Subsequence DTW** finds a template anywhere inside a stream.
 - **Soft-DTW** is differentiable.
 - Speed-ups: LB_Keogh lower bounds, early abandoning.
 - *Your finding:* DTW's cost and accuracy got worse as the number and variability of patterns grew, so you moved to RNNs (1038 ms $\rightarrow$ 14 ms).
- **Fréchet distance** (the "dog-leash" distance; respects order along the curve), **Hausdorff distance** (set-based; ignores order).
- **LCSS / EDR:** robust to noise and outliers because they allow unmatched points.
- **Learned embeddings** (contrastive or siamese networks): similarity = cosine distance in embedding space. Your speakfrench benchmark found Wav2Vec2 embeddings beat DTW-MFCC (ROC-AUC 0.822 vs. 0.599).

6.5 Segmenting motion into steps

- **Change-point detection:** CUSUM, **PELT**, Bayesian online change-point detection (BOCPD).
- **HMMs:** hidden states are steps, emissions are features, and the **Viterbi** algorithm gives the most likely step sequence. **HSMMs** add explicit durations.
- **Finite-state machines** over detected events: your **hierarchical FSM** (Idle $\rightarrow$ Pattern $\rightarrow$ Gesture) at McGill [YOU].
- Learned temporal segmentation (MS-TCN).

6.6 Frequency-domain analysis of motion

- Use the FFT or a spectrogram of wrist trajectories to find **repetitive motion** (repetition rate, a repetitive-strain risk).

- Autocorrelation for **counting repetitions** and estimating cycle time.
- Tremor and vibration show up as high-frequency energy.
- Your DSP background (STFT, S-transform onset detection, MFCC) transfers directly [YOU].

6.7 3D geometry

- **Rotations:**
 - Rotation matrices (SO(3)).
 - **Euler angles** have gimbal lock and discontinuities.
 - **Quaternions** are compact and interpolate smoothly (slerp); q and $-q$ represent the same rotation.
 - Axis-angle.
 - For neural networks, the continuous 6D representation (Zhou et al., 2019) avoids discontinuities.
- **Rigid transforms (SE(3)):** 4x4 homogeneous matrices; chains of frames (facility → cell → fixture).
- **Procrustes / Kabsch:** the best rotation and translation between point sets via SVD. Use it to compare motion with a template *regardless of where the cell is placed*.
- **Geometry queries:** point-in-polygon (zones), distance to a segment or box (hazards), k-d trees and R-trees for fast proximity queries.
- **Orientation from positions:** 2 or more tags on a rigid tool give its orientation.
- **Invariance by design:** relative positions, distances and cell-frame coordinates.

6.8 Ergonomic risk from motion data

- **RULA/REBA** score joint angles, so they need a full pose (from vision or an IMU suit).
- With RTLS badge and wrist tags you get **proxies**:
 - reach distance (wrist ↔ badge)
 - hand height relative to the torso (overhead work, low reaches)
 - repetition frequency
 - static-hold duration
- **The NIOSH lifting equation fits RTLS well:** $RWL = LC \cdot HM \cdot VM \cdot DM \cdot AM \cdot FM \cdot CM$.
 - $LC = 23$ kg.
 - $HM = 25/H$, where H is the horizontal hand distance in cm.
 - $VM = 1 - 0.003 \cdot |V - 75|$, where V is hand height in cm.
 - $DM = 0.82 + 4.5/D$, where D is vertical travel in cm.
 - AM , FM and CM are the asymmetry, frequency and coupling terms.
 - **H, V, D and frequency can be measured from tags.** Load weight comes from MES or bill-of-materials data.
- **Framing to use:** "Start with validated ergonomic standards computed from RTLS proxies, which are explainable and defensible, then learn corrections from expert ratings or incidents." That's physics first, then a learned residual [XH].

6.9 How ultrasonic positioning works (context; ZeroKey's exact method is proprietary)

- **Time of flight:** distance = $c \cdot \Delta t$. **1.5 mm $\approx$ 4.4 μ s** at 343 m/s, which is easy to time. Radio would need about 5 ps for the same distance.
- **Temperature dependence:** $c \approx 331.3 + 0.606 \cdot T(^{\circ}\text{C})$ m/s. A **1 $^{\circ}\text{C}$ error $\approx$ 0.18% $\approx$ 18 mm at 10 m**, which is why self-calibration and compensation matter.

- **Multilateration:** subtract one anchor's range equation from the others to get a linear system, $2(a_i - a_0)^T p = r_0^2 - r_i^2 + \|a_i\|^2 - \|a_0\|^2$. Solve by least squares, then refine with Gauss–Newton. You need 4 or more non-coplanar anchors for an unambiguous 3D position. **Anchor geometry (GDOP)** scales range noise into position error.

6.10 Physics-informed and hybrid modelling [XH: his field is AI for scientific simulation]

- **Hybrid (grey-box) models:** a known physics model plus a learned residual (e.g. KalmanNet, or a kinematic model with a neural correction).
- **PINNs:** loss = data term + λ ·physics residual. **Neural ODEs** for continuous-time dynamics.
- **Constraints as priors:** human speed limits (~2 m/s walking), arm-length limits between wrist and badge (~0.8 m), rigid-body constraints, hand–tool coupling.
- **Surrogate-model lesson:** validate *a posteriori*, inside the full closed-loop system, not only offline. The same applies to detectors: test them in the live pipeline.
- **Simulation for data:** process simulators and digital twins, digital human models, NVIDIA Isaac Sim. Handle the **sim-to-real gap** with domain randomization, fine-tuning on real data, and evaluating on real data only.

7. AI agent basics [minor topic]

- **What an agent is:** an LLM in a loop that **plans, calls tools, observes results and iterates** toward a goal. **ReAct** alternates reasoning and acting.
- **Tool calling:** the model emits structured JSON matching a tool's schema, the runtime executes it, and the result goes back into context.
- **MCP (Model Context Protocol):** an open protocol (JSON-RPC) where servers expose **tools, resources and prompts** to LLM clients. ZeroKey says OmniVisor uses tool calling and MCP.
- **Memory:** short-term (context window), long-term (a vector store or database), scheduled tasks (OmniVisor's "runnable tasks").
- **RAG:** retrieve relevant documents (e.g. SOPs) and put them in the context to ground answers.
- **Key design principle for this role: the LLM should never compute numbers itself.** ML models and analytics are tools the agent calls, which return quantities you can check. The LLM plans, orchestrates and explains.
- **Example flow:** "Why did cell 4 slow down Tuesday?" → `get_cycle_times` → `detect_sequence_deviations` (your model) → `dwell_heatmap` → answer with evidence.
- **Evaluating agents:** a set of real questions with known answers, tool-call accuracy, faithfulness to tool outputs, latency and cost, regression tests on every change.
- **Guardrails:** read-only tools by default, human confirmation before actions, access control over worker-level data, audit logs.
- **LLM basics in case they come up:** decoder-only transformer with next-token prediction; prompting → RAG → fine-tuning; **LoRA** (W + BA, low rank; you used rank 64 on MusicGen [YOU]); hallucination and grounding.
- **VLMs:** a vision encoder plus a projector feeding an LLM (LLaVA-style). CLIP for zero-shot tasks. Possible uses here: reading SOP documents with diagrams, floor plans, or camera frames where they exist.
- **Your honest level:** applied. You build with agentic tools daily (Claude Code) and have fine-tuned foundation models. You aren't an LLM researcher.

8. Reinforcement learning (just in case, not on the recruiter's list)

Glassdoor reported RL questions in other Amii ML Resident rounds, but this email doesn't list it. Know these basics in a sentence each:

- **MDP** (S, A, P, R, γ); policy π ; return $G = \sum \gamma^k r$.
- **Bellman optimality**: $Q^*(s,a) = E[r + \gamma \cdot \max_{a'} Q^*(s',a')]$.
- **TD learning**: $V(s) \leftarrow V(s) + \alpha[r + \gamma V(s') - V(s)]$ (bootstraps: lower variance, some bias). Monte Carlo is unbiased but has high variance.
- **Q-learning** (off-policy, uses the max) vs. **SARSA** (on-policy, uses the next action taken).
- **DQN**: experience replay plus a target network. **Policy gradient**: $\nabla J = E[\nabla \log \pi \cdot (G - b)]$. **Actor-critic**. **PPO**: a clipped policy ratio keeps updates stable.
- **Factories**: you can't explore online, so use **offline RL** (CQL, IQL), off-policy evaluation, **contextual bandits** for recommendations, and simulators.
- **Amii context**: Sutton (Turing Award 2024), *The Alberta Plan*, "The Bitter Lesson." A factory is a **continuing task**, so the average-reward formulation fits.

Part B — The mini case study (10–20 min, interactive)

9. Case-study playbook

9.1 Time plan (for a 15-minute case; stretch or shrink to fit)

Minutes	Step	What to say or do
0–2	Restate and clarify	"Let me make sure I understand the goal..." Ask 3–5 questions (§9.3). Don't start designing before you've clarified.
2–4	Data and assumptions	Sensors or cameras, rates, labels, noise. State your assumptions out loud when they can't answer.
4–6	Baseline	Rules, thresholds or classical methods. "This is what any ML must beat."
6–10	Main approach, plus one alternative	Pipeline diagram, model choice and why, trade-offs.
10–12	Evaluation	Leak-free splits, event-level metrics, error analysis, the success criterion.
12–14	Deployment, risks, next steps	Latency, edge, monitoring, drift, privacy, and what you'd do in week 1.
rest	Their questions	Welcome pushback. It's meant to be interactive.

Check in after each step: "Does that match what you had in mind, or should I go deeper on anything?" This turns it into a conversation, which is what they asked for.

9.2 Tools (optional, so keep it simple)

- **Recommended:** a **blank Excalidraw or tldraw board** (or a Google Doc) open before the interview, ready to share your screen. Pre-draw the empty 7-box skeleton below so you don't waste time on layout.
- **Backup:** pen and paper held up to the camera, or just talking. They said tools aren't required.
- **Don't** build slides during the case, or spend more than 10 seconds on formatting.

[Goal & decision] → [Data & sensors] → [Pre-processing] → [Baseline]
→ [Model(s)] → [Evaluation] → [Deployment & monitoring]
↑ risks / assumptions / open questions (side box)

9.3 Clarifying questions (pick the relevant ones)

1. **What decision does this drive, and who acts on it?** (worker, supervisor, automatic stop, agent)
2. **How fast must it respond?** (real-time alert vs. end-of-shift report) **Which hardware?** (edge gateway, Jetson, server)
3. **Which data do we have?** (RTLS tags and their placement, cameras, IMU, MES/torque events) **At what rate?**
4. **What labels exist?** (incident logs, SOP definitions, expert ratings) **How many positive examples?**
5. **What does a false alarm cost compared with a miss?** Is there an alarm budget?
6. **Scale:** one cell, or many factories? Do we need to generalize to new sites?
7. **Constraints:** privacy (worker data), explainability, safety certification.

9.4 Critical-thinking moves (they're explicitly testing this)

- **State your assumptions** and say how you'd test each one.
- **Put numbers on it:** "at 20 Hz, a 2-second window is 40 samples," "an event rate of 1 in 1,000 means..."
- **Challenge the framing:** Is ML even needed? Could a geofence or rule solve 80% of it? When would you *not* use deep learning?
- **Name failure modes before they do:** occlusion, drift, a new site, label noise, the base-rate problem, alert fatigue.
- **Propose the cheapest experiment** that reduces the most uncertainty: *"Week 1: I'd record a stationary tag to measure real noise, and label one shift with an engineer."*
- **Trade-offs:** accuracy vs. latency, recall vs. false alarms, a general model vs. per-site models, camera vs. RTLS (privacy, coverage, precision).
- **Ethics and adoption:** worker privacy, surveillance concerns, explainability. *"At MAS, adoption depended on operators trusting the system as much as on accuracy."*
- **If they push back:** *"Good point. If that's the case, I'd change X to Y because..."* Adapting well scores better than defending your first idea.
- **If you don't know:** *"I haven't used X directly. Here's how I'd reason about it..."* Then reason from first principles.

9.5 A strong closing (30 seconds)

"To summarize: start with an interpretable baseline and an evaluation harness in the first weeks, move to a streaming model at the edge for the top risks, validate in shadow mode at a pilot site, then extend to unknown anomalies and connect it to the agent. The biggest risk is label scarcity, and I'd tackle it with weak labels from the SOP data and active learning with ZeroKey's engineers."

10. Practice cases (rehearse A, D and E out loud with a timer)

The real case may be RTLS, video, or a general time-series problem. These cover the likely range.

Case A: Risky-behaviour detection on RTLS data, deployed at the edge

- **Clarify:** which risks (ergonomic, human-robot proximity, zone entry, SOP violations), who acts, latency, tag placement, labels, alarm budget.

- **Pipeline:** anchors → position → **per-tag Kalman** (gating, dropout bridging, velocity) → cell frame → streaming features (kinematics, zones, pairwise distances, reach and height) → three tiers:
 1. **Rules and geofences:** hard safety, deterministic.
 2. **Learned classifiers:** causal TCN/GRU, int8.
 3. **Anomaly score:** forecast residuals, for unknown risks. Then post-processing (hysteresis, minimum duration, per-cell thresholds) → alerts and agent tools → feedback loop.
- **Plan (12-month residency):**
 - **Months 0–2:** data audit, label taxonomy, baseline, evaluation harness.
 - **Months 2–5:** streaming models for the top 2–3 risks, sequence-compliance module, self-supervised pre-training, active learning.
 - **Months 5–8:** edge deployment and shadow pilot.
 - **Months 8–12:** anomaly detection, agent integration, knowledge transfer.
- **Evaluation:** event-level precision and recall, false alarms per shift, p95 latency, results by worker, cell and shift.
- **Likely follow-ups:**
 - "No labelled incidents?" → rules, NIOSH proxies, weak labels, active learning.
 - "A new factory?" → cell-frame and invariant features, per-site calibration.
 - "Latency must be 50 ms?" → smaller TCN, int8, streaming state.

Case B: Detecting out-of-sequence bolt tightening

- **Inputs:** tool-tag trajectory (plus wrist tag), bolt positions in the fixture frame, torque-tool events, the SOP order.
- **Pipeline:** Kalman on the tool tag → **nearest-bolt association** at each torque event (a distance threshold set against 1.5 mm noise; resolve ambiguity when bolts are close) → sequence of bolt IDs → **grammar or FSM check** that allows legitimate alternative orders → alert before the next step.
- **Edge cases:** re-torquing, a skipped bolt done later, tool swaps, two workers in one cell, a fixture that isn't where the CAD says (calibrate from data).
- **ML's role:** association under ambiguity, spotting "hesitation" or near-errors, predicting the next likely error.
- **Metrics:** sequence-level accuracy, **false-stop rate** (stopping the line is expensive), latency.
- **Your link:** the McGill hierarchical FSM (event detectors → state machine) [YOU].

Case C: Predicting human-robot near-misses

- Constant-velocity Kalman forecast as a baseline → a learned trajectory forecaster with uncertainty (GRU with Gaussian output, Trajectron-style) → **time-to-collision and minimum predicted distance** against the robot's planned path → graded response (warn, then slow).
- The certified safety system stays deterministic, and ML **adds** early warning on top.
- Metrics: forecast error (ADE/FDE at 0.5–2 s), near-miss recall at a fixed false-warning rate.

Case D: Detecting unsafe lifting from factory video (CV + time series) [likely, given the main topics]

- **Clarify:** camera placement and number, resolution and frame rate, privacy constraints, what "unsafe" means (NIOSH, REBA), labels, real-time vs. offline.
- **Pipeline:** person detection (YOLO) → **tracking** (ByteTrack/DeepSORT, which uses a Kalman filter) → **2D pose** (top-down HRNet/ViTPose, or MediaPipe on the edge) → **temporal smoothing** of keypoints → optional **3D lifting**, or multi-camera triangulation → joint angles → **REBA/NIOSH-style scoring** (interpretable baseline) → **skeleton-based action model** (ST-GCN) to detect lift events and risky postures → alerts and reports.

- **Why pose rather than raw video:** lighter, more robust to clothing and lighting, better for privacy (store skeletons, not video), and it transfers across sites.
- **Failure modes:** occlusion by machinery, a single viewpoint's depth ambiguity (trunk flexion is hard to see side-on vs. front-on), gloves and PPE, several people overlapping, lighting.
- **Evaluation:** pose accuracy on factory frames (PCK), event-level lift detection F1, and agreement of risk scores with ergonomists (Cohen's κ).
- **Critical-thinking point:** *"Would RTLS wrist and badge tags plus NIOSH proxies get 80% of the value without cameras? I'd compare both on a pilot cell. Vision gives full-body posture; RTLS gives precision, no occlusion from lighting, and privacy."*

Case E: Predictive maintenance or anomaly detection from sensor time series (general)

- **Clarify:** what failure means, how many failure examples exist, sensor types and rates (vibration kHz, temperature), how much warning is useful (hours or days).
- **Baseline:** thresholds and control charts (EWMA/CUSUM) on RMS, kurtosis and spectral band energy.
- **Models:**
 - *With labels:* GBDT on window features → 1D CNN or TCN; survival analysis for time-to-failure.
 - *Without labels:* autoencoder or forecasting residuals, isolation forest.
- **Pitfalls:** censored data (machines that haven't failed yet), leakage across machines (split by machine), maintenance events changing the baseline, very few failures.
- **Metrics:** lead time before failure, precision at a maintenance budget, cost-weighted metrics.

Case F: "Should ZeroKey add cameras?" (a pure critical-thinking case)

- **Vision gives:** full-body pose, object identity and appearance, no tags to wear.
- **RTLS gives:** mm precision, no lighting or line-of-sight-to-camera issues, privacy, direct identity through tags.
- **Hybrid options:** use cameras only during **data collection, for labelling**, then deploy RTLS-only. Or fuse them where a customer accepts cameras (Tulip Vision is a precedent).
- **How to decide:** a pilot comparing value added against privacy and deployment cost.

Case G: Agent answers "Why did throughput drop on line 3?"

- Tools with checkable outputs (cycle times, SOP deviations, dwell heatmaps, downtime events) → the LLM plans and explains, citing evidence → evaluation on a set of real questions → guardrails. Keep it short. Agents are a minor topic.

Part C — You

11. Defending your own projects [YOU]

For each project: likely probes, and where to fill in real details. **Wherever it says [fill in], check what you actually did before the interview. Don't improvise facts.**

11.1 JAES 2026: real-time audio pattern detection

- **Summary:** MFCC features (30 ms windows, 10 ms hop) → stacked RNN trained on rule-based synthetic variations (about 10k per pattern). RPi4: **14 ms, 31.4% CPU, F1 0.76** vs. DTW's **1038 ms, 48.9% CPU, F1 0.65. 74× faster.**
- "Is F1 0.76 good?" The benchmark is polyphonic, performed by live musicians with expressive variation (DoPP: 20 performers, 176 patterns). The model beat DTW on both accuracy and latency. [fill in: main failure modes]
- "How did you choose the threshold?" [fill in]
- "Didn't the model just learn the synthetic generator?" Evaluation used real human recordings. [fill in: whether any performers were held out]
- "What does the 14 ms include?" [fill in: feature extraction plus inference? warm-up? number of runs?]
- "What would you change now?" Build the benchmark harness earlier, try a TCN or small causal transformer, add quantization.

11.2 McGill smart guitar (IMU + audio gesture fusion)

- **Summary:** BNO055 IMU at 100 Hz, single LSTM-256 on a 1 s accelerometer buffer, **hierarchical FSM** (Idle → Pattern → Gesture) fusing gesture and audio detections. **F1 0.78** on 500 events (P 0.79 / R 0.76). Case study with 5 guitarists.
- **This is your most direct motion-analysis evidence.** Expect: "How did you segment gestures?" "What features did the IMU provide?" "How did you handle orientation and gravity?" [fill in: raw accelerometer vs. the chip's fused orientation; gravity removal; windowing]
- "Why an FSM instead of end-to-end fusion?" Few labels, interpretability, and it composes detectors you've validated separately. The same pattern works for SOP compliance.

11.3 MUSMET: multimodal stream sync (4 musicians × audio + EEG + MR head/hand tracking)

- **Your closest evidence for multi-entity 3D motion data.**
- "How did you synchronize clocks?" [fill in]
- "What were the tracking rates, and how did you handle drift and dropped samples?" [fill in]
- "What did you do with the head and hand trajectories?" [fill in]

11.4 MAS industrial CV (the "practical industry CV" topic; Xu Han has done similar work)

- **Care-label QC:** OpenCV template and feature matching with an explainable overlay. Three iterations (scanner → industrial camera → conveyor with PLC reject). **15–20 min per label → 1–2 min per batch.**
- **Fabric defects:** microscopic camera array, overlapping fields of view across 60-inch rolls, **5–6 s alert budget**, a 50% operator reduction in the pilot.
- **Sewing-floor IoT:** tracking operators' repetitive motion patterns for productivity. **That's motion analysis in a factory.** [fill in: what the sensors measured]
- "Why classical CV, not deep learning?" Limited data and compute at the time, explainability for QA, deterministic behaviour for the PLC. "Today I'd consider PatchCore-style anomaly detection for open-ended defect types."
- "How did you get operators to adopt it?" Explainable overlay, human-in-the-loop fallback, iterative rollout.

11.5 Promptly

- Geometry-alignment algorithm (camera-captured garment geometry → image warp per side), **RIP integration you built yourself**, consulting on the flipping mechanism. Now a product with facilities in four countries. Details are under **NDA. Never mention the patent.**

- Link: applied geometric CV (registration and warping).

11.6 speakfrench

- **100,000 scored trials; Wav2Vec2 XLSR-FR ROC-AUC 0.822 vs. DTW-MFCC 0.599.** Evidence that learned embeddings beat templates when variation is high. Raise the caveat yourself: TTS voices aren't real learners.

11.7 VarianceEngine

- LoRA (rank 64) on MusicGen-medium with a custom cross-attention conditioning head, two GPUs. **Not yet evaluated.** Say so. Comparing five architectures before choosing is good evidence of structured model selection.

12. Question bank with short answers (spoken, no code)

ML and statistics [main]

1. *Bias-variance?* The error decomposition (§1.1); learning curves to diagnose.
2. *L1 vs. L2?* Laplace vs. Gaussian priors; sparsity vs. shrinkage.
3. *MLE vs. MAP?* MAP = MLE with a prior; priors act as regularization.
4. *What is a p-value?* The probability of data at least this extreme if the null hypothesis is true. Not the probability the null is true.
5. *Type I vs. Type II error in a safety detector?* A false alarm vs. a missed hazard. Which matters more depends on cost; safety usually favours recall within an alarm budget.
6. *How do you compare two models fairly?* Same leak-free splits, paired tests (McNemar, Wilcoxon), bootstrap confidence intervals (block bootstrap for time series).
7. *Why is autocorrelation a problem?* Fewer effectively independent samples, confidence intervals that are too narrow, leakage in random splits.
8. *ROC vs. PR curve?* Use PR under imbalance.
9. *Base-rate fallacy?* §1.2. Rare events mean mostly false alarms, even with a good detector.
10. *Explain PCA.* Eigenvectors of the covariance matrix, i.e. the directions of maximum variance, computed via SVD.
11. *Bagging vs. boosting?* Bagging reduces variance (random forest); boosting reduces bias (GBDT).
12. *Did the intervention reduce risk? How would you know?* Control cells, difference-in-differences, a confounder check, statistical power.
13. *Handling class imbalance?* §1.3.
14. *Calibration: why and how?* Scores should mean probabilities for decisions and agents; use temperature scaling and reliability diagrams.

Deep learning for time series [main] 15. *LSTM vs. GRU vs. TCN vs. transformer?* §3.1 table; choose by latency, data size and context length. 16. *Why $\sqrt{d_k}$ in attention?* Keeps softmax out of saturation. 17. *How do transformers know order?* Positional encodings; time embeddings for irregular sampling. 18. *Few labels?* Self-supervised learning, weak supervision, active learning, synthetic data. 19. *Streaming vs. offline models?* Causal models with state vs. bidirectional ones; latency and group delay. 20. *Time-series foundation models?* Good zero-shot forecasting baselines; not specialized for 3D multi-entity motion.

Computer vision [main] 21. *CNN vs. ViT?* Inductive bias vs. flexibility; ViT needs pre-training or lots of data. 22. *One-stage vs. two-stage detection? What's NMS?* §4.2. 23. *Top-down vs. bottom-up pose?* Accuracy vs. scaling with the number of people. 24. *How do you recognize actions in video?* Two-stream, 3D CNN, SlowFast/TSM, video transformers, or pose → ST-GCN. 25. *How does multi-object tracking work?* Kalman prediction + Hungarian matching

(SORT), plus appearance features (DeepSORT). 26. *Defect detection with almost no defect images?* Anomaly detection trained on good parts (PatchCore, PaDiM), plus careful lighting. 27. *Video data leakage?* Split by clip, camera or site, never by frame. 28. *Camera vs. RTLS for monitoring motion?* §10 Case F. 29. *What is optical flow?* Pixel motion between frames; brightness constancy; Lucas–Kanade, RAFT. 30. *Pinhole model?* $x = K[R|t]X$.

Practical industry [main] 31. *Walk me through an industrial ML project.* §5.1 lifecycle. 32. *The model works in the lab and fails in the factory. Why?* Distribution shift, leakage, label mismatch, latency, sensor placement. Fix with shadow pilots and per-site data. 33. *How do you reduce alert fatigue?* Event-level thresholds, hysteresis, alarm budgets, operator feedback. 34. *Too slow on the device?* Profile → streaming state → smaller architecture → distillation → quantization → runtime tuning. 35. *Monitoring after deployment?* Drift, alert rates, feedback, latency, retraining triggers.

Motion math [headline] 36. *Explain the Kalman filter.* §6.3. 37. *How do you tune Q and R?* R from spec or a stationary tag; Q from motion physics; check NIS consistency. 38. *Why not just differentiate positions?* §6.1 noise amplification, with numbers. 39. *DTW vs. Fréchet vs. learned embeddings?* §6.4. 40. *How do you segment motion into steps?* Change points, HMM/Viterbi, FSMs, MS-TCN. 41. *How do you detect repetitive motion?* FFT or autocorrelation of trajectories. 42. *Quaternions vs. Euler angles?* Gimbal lock and discontinuities vs. smooth interpolation. 43. *Compare a motion to a template anywhere in the cell?* Procrustes/Kabsch alignment, then a distance measure. 44. *Ergonomic risk from a few tags?* NIOSH proxies (§6.8).

Agents [minor] 45. *What's an AI agent?* An LLM plus a planning loop plus tools plus memory. 46. *What is MCP?* A standard protocol that exposes tools, resources and prompts to LLM clients. 47. *How would your models fit into OmniVisor?* As tools returning quantities you can check. 48. *Evaluating an agent?* A task set, tool-call accuracy, faithfulness, latency.

About you [YOU] 49. *The hardest real-time problem you've solved?* [fill in: one true story] 50. *A project that changed direction?* The DTW → RNN pivot, or the care-label QC iterations.

13. Gap scripts (brief, confident, then move on)

Gap	Script
Modern deep CV (detection, pose)	"My production CV was classical and industrial: template and feature matching, registration and warping, camera arrays wired into PLCs. I know the modern stack (YOLO, HRNet/ViTPose, ST-GCN, PatchCore) and how I'd use it here. My deep-learning depth is on the temporal side, which is where the pose-based analysis happens."
Kalman in production	"I'm fluent with the maths and the standard variants. My hands-on state estimation was IMU-based, and that sensor did its fusion on the chip. So I'd apply well-understood filters here and validate them with static-tag and known-trajectory tests." (Only the parts that are true.)
3D reconstruction / simulation	"I haven't built reconstruction pipelines. My 3D work is tracking, registration and alignment. It's where I'd lean on Amii's scientists."
RL	"I know the fundamentals and I haven't deployed RL. For a factory I'd start with bandits or offline RL."
LLM research	"Applied level. My value is making the perception models reliable tools for the agent."
PyTorch vs. TF	"My published work is TensorFlow/Keras. My recent projects are PyTorch. I'm comfortable in both." (Less important now that there's no coding.)

14. Questions for Xu Han (pick 2–3 for the end)

1. "How did you and ZeroKey scope this project, and what does success look like at month 12?"
2. "Which risk behaviours are the top priority, and what labels exist today?"
3. "What's the edge target and latency budget, and what do the noise and dropout characteristics look like in a real factory?"
4. "Does the project involve any camera or video data, for labelling or fusion, or is it RTLS only?" (This explains why the JD lists CV.)
5. "Is there room for physics-informed priors or simulation, for example kinematic constraints or synthetic trajectories?" [XH]
6. "From your CV quality-inspection projects, what lessons about deploying on the floor would carry over here?" [XH]
7. "How is the residency run day to day: research versus delivery, and how often you meet with ZeroKey?"

(Some of these may already come up in the case study. If so, skip them and ask about the residency itself.)

15. Final 24–48 hours

- ☐ **Rehearse Cases A, D and E out loud**, 15 minutes each, following the §9.1 time plan. Record one and listen back.
 - ☐ Set up the **Excalidraw/tldraw board** with the 7-box skeleton and test screen sharing on the interview platform.
 - ☐ Explain these **out loud, without notes, in about 60 seconds each**: the Kalman filter, bias–variance, p-value, PCA, attention, top-down vs. bottom-up pose, SORT tracking, DTW, PTQ vs. QAT, what MCP is.
 - ☐ Fill in every **[fill in]** in §11 with true details.
 - ☐ Have numbers ready: **14 ms / F1 0.76 / 74x / 31.4% CPU; F1 0.78 gesture fusion; ROC-AUC 0.822; 15–20 min → 1–2 min per batch; 1.5 mm $\approx$ 4.4 μ s; 1 °C $\approx$ 18 mm at 10 m; velocity noise $\approx$ 42 mm/s at 20 Hz.**
 - ☐ Skim one Xu Han paper (check the authorship first) and ZeroKey's OmniVisor page.
 - ☐ Setup: quiet room, stable connection, pen and paper, water.
-

Appendix A — Reference code (for understanding only; there's no coding in this interview)

Reading these helps you explain the methods precisely. You won't be asked to write them.

Kalman filter (constant velocity, 3D, with gating)

```

import numpy as np

def make_cv_model(dt, sigma_a, sigma_z):
    I3, Z3 = np.eye(3), np.zeros((3, 3))
    F = np.block([[I3, dt * I3], [Z3, I3]])
    H = np.hstack([I3, Z3])
    G = np.vstack([0.5 * dt**2 * I3, dt * I3])
    Q = sigma_a**2 * G @ G.T          # white-noise acceleration
    R = sigma_z**2 * I3              # e.g. sigma_z = 1.5e-3 m
    return F, H, Q, R

def kf_step(x, P, z, F, H, Q, R, gate=11.34):  # chi2(3 dof), 99%
    x, P = F @ x, F @ P @ F.T + Q          # predict
    if z is None:                            # dropout: predict only
        return x, P, False
    y = z - H @ x                          # innovation
    S = H @ P @ H.T + R
    if y @ np.linalg.solve(S, y) > gate:    # outlier, e.g. multipath
        return x, P, False
    K = P @ H.T @ np.linalg.inv(S)
    return x + K @ y, (np.eye(len(x)) - K @ H) @ P, True

```

DTW (with an optional Sakoe–Chiba band)

```

def dtw(a, b, band=None):                  # a: (n, d), b: (m, d)
    n, m = len(a), len(b)
    D = np.full((n + 1, m + 1), np.inf); D[0, 0] = 0.0
    for i in range(1, n + 1):
        lo, hi = (1, m) if band is None else (max(1, i - band), min(m, i + band))
        for j in range(lo, hi + 1):
            cost = np.linalg.norm(a[i - 1] - b[j - 1])
            D[i, j] = cost + min(D[i - 1, j], D[i, j - 1], D[i - 1, j - 1])
    return D[n, m]

```

Multi-head self-attention (PyTorch)

```

import math, torch, torch.nn as nn

class MultiHeadAttention(nn.Module):
    def __init__(self, d_model, n_heads):
        super().__init__()
        self.h, self.d_k = n_heads, d_model // n_heads
        self.qkv, self.out = nn.Linear(d_model, 3 * d_model), nn.Linear(d_model, d_model)

    def forward(self, x, mask=None):        # x: (B, T, d_model)
        B, T, _ = x.shape
        q, k, v = self.qkv(x).view(B, T, 3, self.h, self.d_k).permute(2, 0, 3, 1, 4)
        scores = q @ k.transpose(-2, -1) / math.sqrt(self.d_k)
        if mask is not None:
            scores = scores.masked_fill(mask == 0, float("-inf"))
        y = (scores.softmax(-1) @ v).transpose(1, 2).reshape(B, T, -1)
        return self.out(y)

```

Sources

- Recruiter format email (2026-09-28): pasted into session by Nishal
- Job posting snapshot (2026-08-25): `career-ops/.playwright-mcp/page-2026-08-25T18-29-00-772Z.yml`
- Glassdoor, Amii ML Resident interviews: https://www.glassdoor.com/Interview/Amii-Canada-Machine-Learning-Resident-Interview-Questions-EI_IE4507580.0,11_KO12,37.htm
- Xu Han profile: <https://www.amii.ca/people/xu-han> · Scholar: <https://scholar.google.com/citations?hl=en&user=PEFI4WsAAAAJ>
- ZeroKey Quantum RTLS: <https://zerokey.com/quantum-rtls/> · OmniVisor AI: <https://zerokey.com/omnivisor-ai/>
- OmniVisor RL + LLM + MCP: <https://metrology.news/zerokey-marries-3d-location-accuracy-with-ai-to-redefine-factory-measurement-intelligence/>
- Candidate profile and verified numbers: `ai-job-search/input/knowledge-graph.md`
- Standard literature referenced: Sutton & Barto (2nd ed.); Bewley et al., "SORT" (2016); Yan et al., "ST-GCN" (2018); Feichtenhofer et al., "SlowFast" (2019); Lin et al., "TSM" (2019); Sun et al., "HRNet" (2019); Xu et al., "ViTPose" (2022); Pavlo et al., "VideoPose3D" (2019); Roth et al., "PatchCore" (2022); Carion et al., "DETR" (2020); Teed & Deng, "RAFT" (2020); Farha & Gall, "MS-TCN" (2019); Nie et al., "PatchTST" (2023); Revach et al., "KalmanNet" (2022); Zhou et al., rotation continuity (2019); Waters et al., revised NIOSH lifting equation (1993).